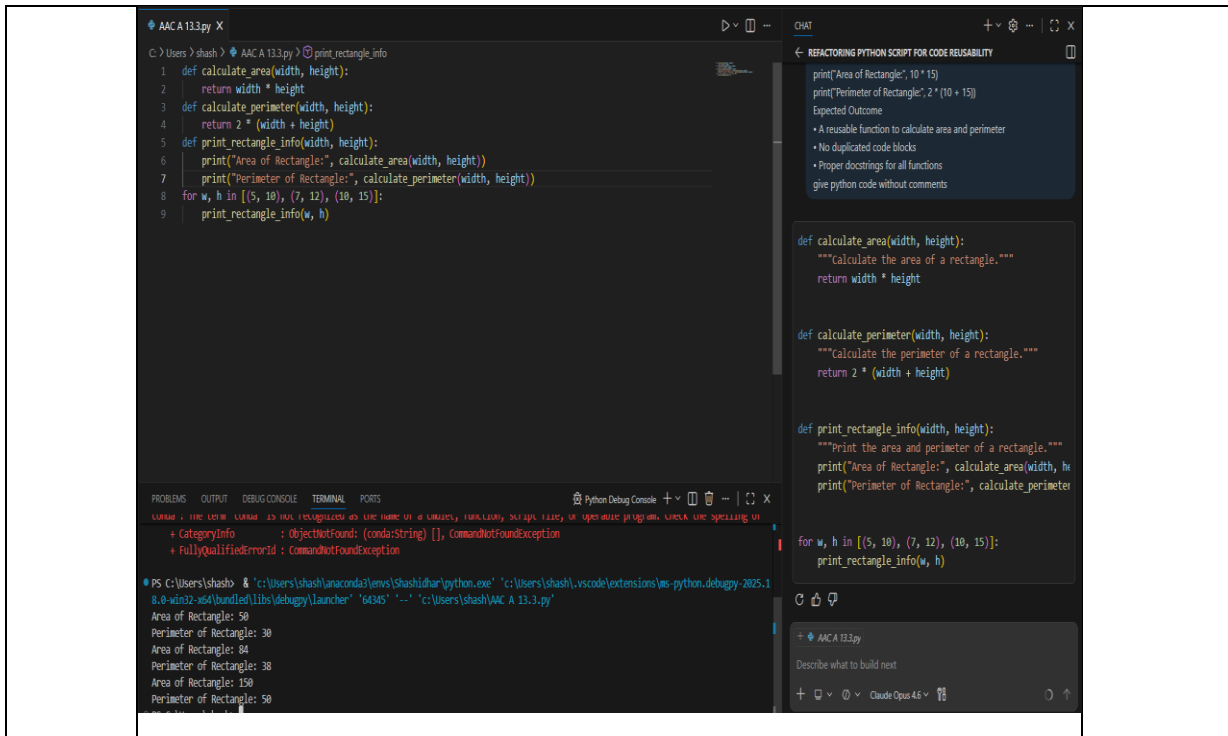


Name: J. Mukhesh H.No: 2303A51813s Batch: 26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M. Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
Mr. Bingi Raju			
Course Code	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week 7 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number: 13.3 (Present assignment number) / 24 (Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 13: Code Refactoring Using AI Assistance Improving Legacy Code for Readability, Maintainability, and Performance Lab Objectives The objectives of this laboratory exercise are to:	Week 7 - Wednesday	

	- Identify common code smells and inefficiencies in legacy Python programs. - Use AI-assisted coding tools to refactor code for better readability and maintainability. - Apply modern Python best practices while preserving original program behavior. - Analyze performance improvements achieved through refactoring.	
	Learning Outcomes After completing this lab, students will be able to: - Detect and eliminate code duplication in Python programs. - Refactor inefficient logic using optimized data structures and constructs. - Improve code quality through meaningful naming conventions and documentation. - Validate refactored code using test cases and performance comparison.	
	Task 1: Refactoring – Removing Code Duplication Objective To eliminate repeated logic by extracting reusable functions. Task Description Use AI assistance to refactor a legacy Python script that contains repeated blocks of code calculating the area and perimeter of rectangles. Starter (Legacy) Code <pre># Legacy script with repeated logic print("Area of Rectangle:", 5 * 10) print("Perimeter of Rectangle:", 2 * (5 + 10)) print("Area of Rectangle:", 7 * 12) print("Perimeter of Rectangle:", 2 * (7 + 12)) print("Area of Rectangle:", 10 * 15) print("Perimeter of Rectangle:", 2 * (10 + 15))</pre> Expected Outcome - A reusable function to calculate area and perimeter - No duplicated code blocks - Proper docstrings for all functions	



Task 2: Refactoring – Optimizing Loops and Conditionals

Objective

To improve performance by replacing inefficient nested loops with optimized structures.

Task Description

Use AI to analyze and refactor a script that checks the presence of elements using nested loops.

Starter (Legacy) Code

```
names = ["Alice", "Bob", "Charlie", "David"]
```

```
search_names = ["Charlie", "Eve", "Bob"]
```

```
for s in search_names:
```

```
    found = False
```

```
    for n in names:
```

```
        if s == n:
```

```
            found = True
```

```
    if found:
```

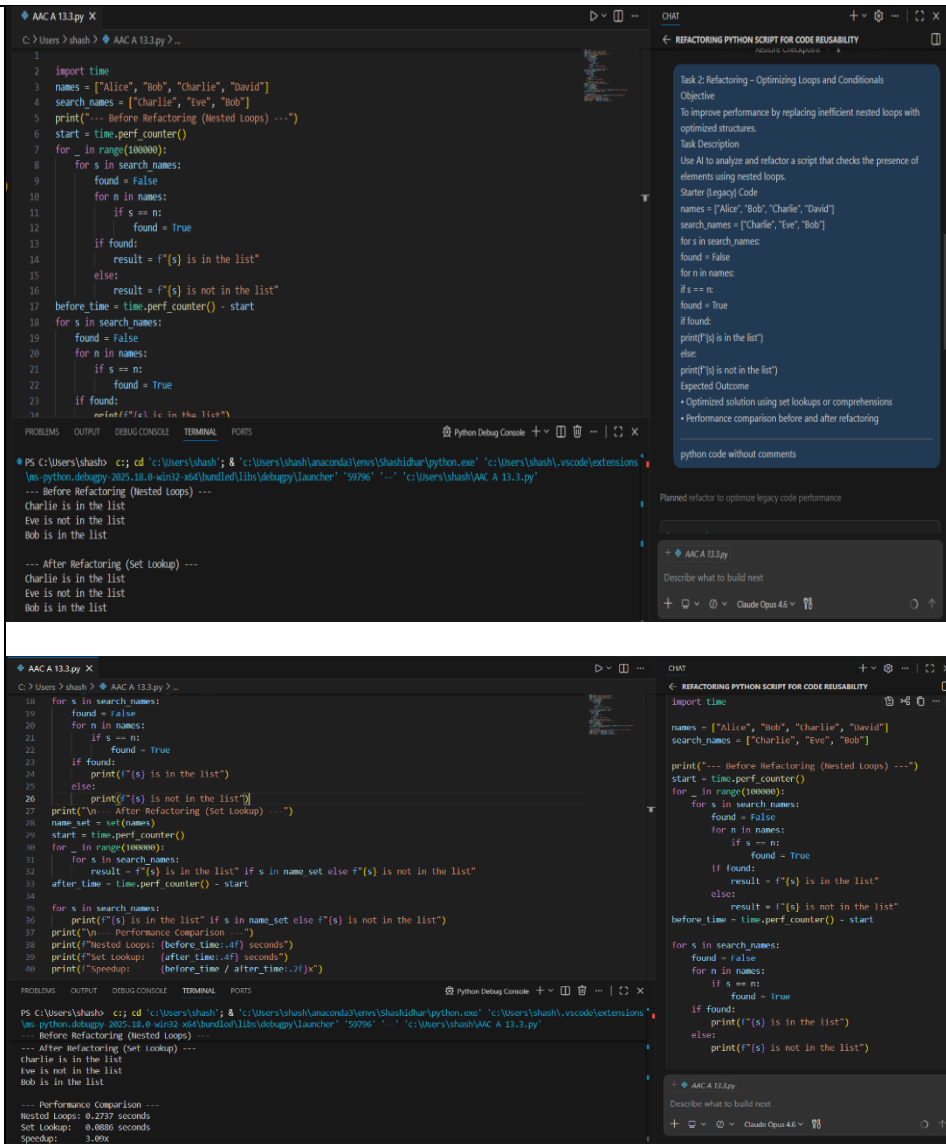
```
        print(f'{s} is in the list')
```

```
    else:
```

```
        print(f'{s} is not in the list')
```

Expected Outcome

- Optimized solution using set lookups or comprehensions
- Performance comparison before and after refactoring



Task 3: Refactoring – Extracting Reusable Functions

Objective

To modularize code by extracting calculations into reusable functions.

Task Description

Refactor a legacy script where price and tax calculations are written inline.

Starter (Legacy) Code

price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)

Expected Outcome

- A function calculate_total(price)
- Cleaner and reusable code structure
- Proper documentation

```
C:\Users\shash > shash > AAC A 13.3.py > calculate_total
1 def calculate_total(price, tax_rate=0.18):
2     tax = price * tax_rate
3     return price + tax
4
5 for price in [250, 500]:
6     print("Total Price:", calculate_total(price))

--- Before Refactoring (Nested Loops) ---
--- Performance Comparison ---
Nested Loops: 0.2737 seconds
Set Lookup: 0.0886 seconds
Speedup: 3.89x
PS C:\Users\shash> cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '59796' '-...' 'c:\Users\shash\AAC A 13.3.py'
Total Price: 295.0
Total Price: 590.0
```

REFACTORING PYTHON SCRIPT FOR CODE REUSABILITY

```
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
Expected Outcome
• A function calculate_total(price)
• Cleaner and reusable code structure
• Proper documentation
python code without comments

def calculate_total(price, tax_rate=0.18):
    """calculate the total price including tax."""
    tax = price * tax_rate
    return price + tax

for price in [250, 500]:
    print("Total Price:", calculate_total(price))
```

Python Debug Console

```
PS C:\Users\shash> cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '62556' '-...' 'c:\Users\shash\AAC A 13.3.py'
Total Price: 295.0
Total Price: 590.0
```

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Objective
To improve maintainability by replacing magic numbers with named constants.

Task Description

Use AI to identify hardcoded values and replace them with constants.

Starter (Legacy) Code

```
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Expected Outcome

- Constants such as PI and RADIUS
- Cleaner, maintainable code

```
C:\Users\shash > shash > AAC A 13.3.py > ...
1 import math
2 PI = math.pi
3 RADIUS = 7
4 print("Area of Circle:", PI * (RADIUS ** 2))
5 print("Circumference of Circle:", 2 * PI * RADIUS)

--- Before Refactoring (Nested Loops) ---
--- Performance Comparison ---
Nested Loops: 0.2737 seconds
Set Lookup: 0.0886 seconds
Speedup: 3.89x
PS C:\Users\shash> cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '59796' '-...' 'c:\Users\shash\AAC A 13.3.py'
Total Price: 295.0
Total Price: 590.0
PS C:\Users\shash> cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '53649' '-...' 'c:\Users\shash\AAC A 13.3.py'
Area of circle: 153.93804002589985
Circumference of Circle: 43.982297150257104
```

REFACTORING PYTHON SCRIPT FOR CODE REUSABILITY

Task Description
Use AI to identify hardcoded values and replace them with constants.

Starter (Legacy) Code
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
Expected Outcome
• Constants such as PI and RADIUS
• Cleaner, maintainable code
python code without comments

```
import math

PI = math.pi
RADIUS = 7

print("Area of circle:", PI * (RADIUS ** 2))
print("Circumference of circle:", 2 * PI * RADIUS)
```

Python Debug Console

```
PS C:\Users\shash> cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '53649' '-...' 'c:\Users\shash\AAC A 13.3.py'
Area of circle: 153.93804002589985
Circumference of Circle: 43.982297150257104
```

Task 5: Refactoring – Improving Variable Naming and Readability Objective

To enhance readability using descriptive variable names and comments.

Task Description

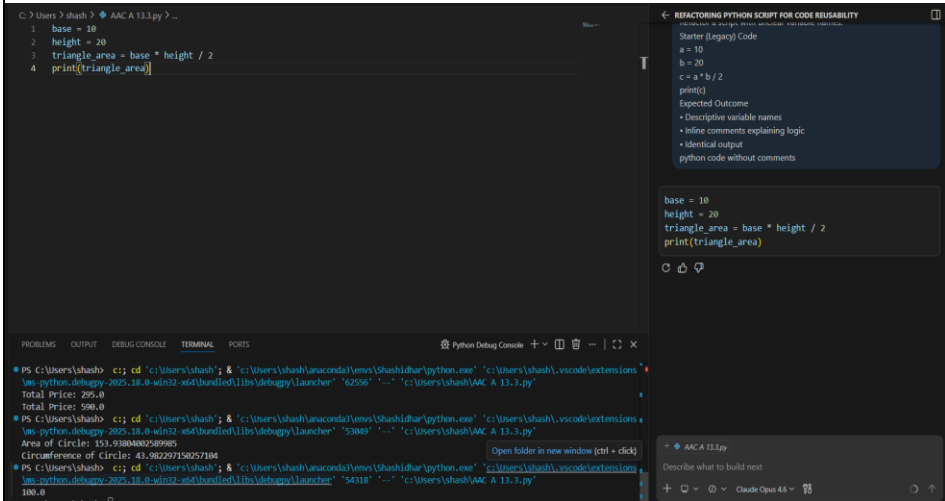
Refactor a script with unclear variable names.

Starter (Legacy) Code

```
a = 10
b = 20
c = a * b / 2
print(c)
```

Expected Outcome

- Descriptive variable names
- Inline comments explaining logic
- Identical output



Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.